

CLIPPER Beyond Shortlisting: Auditable Decision Support for Changing Municipal Micromobility Policies

Author preprint. Accepted for presentation at SpatialConnect '26, co-located with ACM SIGSPATIAL 2026.

Julian Teusch
julian.teusch@tu-clausthal.de
Institute of Computer Science
Clausthal University of Technology
Clausthal-Zellerfeld, Germany

Jörg Philipp Müller
joerg.mueller@tu-clausthal.de
Institute of Computer Science
Clausthal University of Technology
Clausthal-Zellerfeld, Germany

Monika Sester
monika.sester@ikg.uni-hannover.de
Institute of Cartography and
Geoinformatics
Leibniz University Hannover
Hannover, Germany

Abstract

In municipal planning workshops, planners and other stakeholders compare shared-micromobility parking policies by varying no-parking zones, retained sites, spacing, or area allocations. Each edit changes feasible sites and how much demand they cover, so the alternative must be reoptimized on the same spatial data. Full-set greedy, the transparent reference for this task, takes tens of seconds per alternative at city scale. We present Constraint-exact Low-latency Iterative Planning with Pooled Evaluation and Replay (CLIPPER), an optimizer with audit functions developed for requirements elicited with the City of Braunschweig. In each greedy round, it forms a deterministic candidate pool of bounded size, computes how much still-uncovered demand each candidate would add, and rejects candidates that violate an active constraint. An optional offline audit scans every remaining feasible candidate and records what the restricted pool omitted. We evaluate these functions on complete eleven-state edit chains ($E_0, \dots, E_{10}$) in Braunschweig, Munich, and Berlin. With $K = 1024$ candidates per group, the fixed-width mode CLIPPER-F has mean coverage gaps to full-set greedy under the same policy of 0.245, 0.003, and 0.001 percentage points in Braunschweig, Munich, and Berlin, respectively, while mean rollout time falls by factors of 13.6–28.9; no audited run terminates while a candidate outside the pool could still increase coverage. Plans computed from two checksummed versions of Braunschweig’s official no-parking-zone data differ in 30 of about 540 selected sites although coverage moves by only about 0.1 percentage points. These changes still require municipal assessment and implementation. The findings inform a proposed municipal process that versions policy inputs, reports site changes beside coverage, and scans the full candidate set before a final decision.

CCS Concepts

• **Information systems** → **Decision support systems**; • **Theory of computation** → *Submodular optimization and polymatroids*; • **Applied computing** → *Transportation*.

Keywords

spatial decision support, facility location, submodular coverage, audibility

1 Introduction

Shared micromobility can support first/last-mile access, but without deliberate parking and station planning it also creates curbspace and compliance problems [9, 22, 28]. A municipality designing shared-scooter parking must choose a limited number of zones so that as many trips as possible begin or end within walking distance of one. We call each possible zone location a *candidate*. Policy rules narrow the choice. *Exclusions* close areas to new parking, *locks* retain existing sites, and *spacing* keeps new zones a minimum walking distance apart. *Caps* distribute the facility budget across areas by limiting how much any one area may receive.

Workshop-based comparison turns these rules into alternatives: municipal planners and other stakeholders vary them, inspect a feasible plan after each edit, and compare coverage and site changes. Braunschweig’s municipal quality agreement permits riding and parking restrictions to be revised at any time, including temporary no-return areas for major events [23]. Our collaboration with the City established requirements to revise restrictions, retain mandatory sites, enforce spacing and area allocations, and compare feasible alternatives over common data. We encode these requirements in the scenario model. Each edit changes admissible candidates, so the previous plan can become infeasible or cover less demand. The task is therefore not one-off facility placement, but repeated comparison of policy alternatives on a common spatial basis.

Comparison must also reach below aggregate numbers. In our documented policy-input case, two official versions of Braunschweig’s no-parking-zone data, each identified by a checksum, produce plans that differ in 30 of about 540 selected sites although aggregate coverage moves by only about 0.1 percentage points (Table 5). A municipality would still have to assess, implement, and communicate every such change, so decision support for changing policies must version its inputs and report which selected sites change, not only how much demand is covered.

From one optimization run to repeated comparison. Planning-support research treats rapid “what-if” exploration as an aid to deliberation rather than a replacement for planning judgment [10, 17, 18]. A repeated edit–solve–compare loop creates four computational requirements. Every returned plan must satisfy the active spatial and policy constraints. Solving each alternative must be fast enough to make a sequence of comparisons practical. If the optimizer considers only part of the candidate set, the effect of that restriction must be measurable. Finally, stored scenarios

and deterministic execution must allow a result to be reproduced and compared later. We evaluate these properties on complete, controlled scenario chains and on documented municipal policy inputs.

Technical gap. Maximal covering and monotone submodular optimization provide transparent siting models [5, 16]. *Full-set greedy* repeatedly adds the feasible candidate with the largest current coverage gain. We use it as the reference because it applies the same transparent selection rule to the complete candidate set under the same constraints and caps, thereby isolating the effect of restricting the pool on coverage. In our benchmarks, this control averages 23–53 seconds per state, so all eleven states require 4.2–9.7 minutes of solver time before inspection or discussion. Lazy, stochastic, and thresholded greedy methods reduce the number of gain calculations under their respective models [2, 12, 15]; dynamic and prediction-augmented methods maintain solutions across update sequences [1]. CLIPPER solves every stored state from the beginning. Its selector enforces locks, exclusions, area caps, conflict classes that prevent duplicate use of the same curb space, and a minimum network distance between sites. A replaceable rule limits the candidates considered in each round. Within that pool, the selector computes exact current coverage gains and checks every active constraint before adding a site.

Contribution. CLIPPER narrows *what the optimizer looks at* without relaxing *what it enforces*. We contribute Constraint-exact Low-latency Iterative Planning with Pooled Evaluation and Replay (CLIPPER) as an optimizer with audit functions for municipal decision support. Its contributions are:

- **Exact enforcement of the recorded policy.** The selector starts with mandatory locks and adds only candidates that satisfy the active exclusions, spacing rule, conflict classes, area caps, and global budget. A fixed ordering rule resolves ties, so the same recorded inputs produce the same selections.
- **Two explicit ways to form the candidate pool.** In the fixed-width mode CLIPPER-F(K), every active proposal group offers up to K candidates under balanced area caps. The adaptive-budget mode CLIPPER-A(B_{prop}) distributes one total candidate budget among the groups and uses a stated policy that gives more capacity to areas with more active candidates. Each mode is compared with full-set greedy under that mode’s policy.
- **Separate checks during planning and after a run.** A lightweight score can request a larger pool or a later audit. The exact offline audit instead scans every remaining feasible candidate and records how much additional coverage the restricted pool omitted in each round.
- **Municipally grounded requirements and policy evidence.** Two versions of the city’s official no-parking zones show how revised exclusion geometry changes both feasibility and the selected sites; each version is identified by a checksum [24]. These results support a proposed municipal process from registering policy data and calibrating the candidate rule to comparing alternatives, making a final decision, and archiving its audit (Section 6).

Extension beyond the four-page report. An accepted four-page SIGSPATIAL paper introduced CLIPPER and the primary comparisons over the eleven-state chains [26]. Beyond that short paper, this article adds four elements: audits that scan all remaining candidates at three values of K , a comparison rule that updates scores only for candidates affected by newly covered demand, an evaluation using two official versions of municipal policy geometry, and the implementation process in Section 6. We test the optimization procedure on all eleven states ($E_0, \dots, E_{10}$) of controlled three-city chains and examine revised municipal geometry in the Braunschweig case.

We evaluate these technical functions through three questions, then set out their proposed use in municipal planning (Section 6):

- Q1 How much time does the restricted pool save under the same policy, how large is the remaining coverage gap, and does the returned plan satisfy every encoded constraint?
- Q2 How often does a pool ordered once per state omit a larger feasible gain, and how do score updates after each selection change coverage and runtime?
- Q3 How much can two official versions of exclusion geometry change the selected sites when their aggregate coverage is similar?

For Q1, CLIPPER-F(1024) has a mean coverage gap over the eleven states of 0.245 percentage points in Braunschweig, 0.003 in Munich, and 0.001 in Berlin, and cuts mean rollout time by factors of 13.6–28.9. Under its coverage-prioritized policy, CLIPPER-A(8192) differs from the corresponding full-set greedy run by 1.82 percentage points in Braunschweig, 0.12 in Munich, and 0.27 in Berlin, while using 9–15% of that run’s rollout time (Table 3 and Section 5). Q2 and Q3 are answered by the round-by-round audit, score-update comparison, and municipal policy-input case in Tables 4 and 5. We additionally verify deterministic replay as a system property in Section 5.2.4.

2 Related work

Planning support and repeated comparison. Planning support systems organize “what-if” analysis, communication about scenarios, and stakeholder deliberation [7, 10]. Research on their adoption emphasizes usability, trust, and transparent trade-offs rather than autonomous replacement of planning judgment [17–19]. CLIPPER supplies the computational part of such a workflow: it solves each recorded policy state independently, retains the information needed to compare that result later, and measures the effect of restricting the candidate pool.

Reducing work in greedy submodular optimization. Maximal covering and monotone submodular maximization provide the objective foundation [5, 16]. Standard greedy repeatedly selects the feasible candidate with the largest current gain. Lazy evaluation keeps the full candidate set but uses valid upper bounds to avoid recalculating every gain. Stochastic greedy samples candidates in each round. Thresholded methods skip candidates whose gains fall below a decreasing threshold [2, 12, 15]. These approaches reduce the number of gain calculations under their stated feasibility models. CLIPPER limits the set passed to the selector, calculates exact current gains within that set, and checks the combined scenario

constraints before every insertion. The evaluation includes full-set, stochastic, thresholded, and local-search controls. Its primary comparisons use full-set greedy under the same policy as the corresponding CLIPPER mode.

Optimization under changing inputs. Dynamic submodular algorithms maintain a solution as elements or objective information change; learning-augmented variants use forecasts to reduce the work needed for an update [1]. Agarwal and Balkanski study updates under a cardinality constraint, whereas the planning states here combine locks, caps for individual areas, conflict classes, exclusions, and network-distance spacing. CLIPPER discards the previous solution after a policy edit and solves the recorded state from the beginning; Section 5 additionally compares candidate orders that do and do not update scores during a solve.

Facility location for micromobility. Station-location studies for bicycle sharing use geographic information systems (GIS) and demand data [6], including crowd-planned station redeployment in SIGSPATIAL [29]. More recent work considers geofenced parking identification, station planning, and time-varying curb allocation [4, 21, 30]. Reinforcement learning has also been applied to strategic geofenced facility planning [27]. CLIPPER focuses on deterministic recomputation of weighted coverage plans under an explicit policy state.

Learning for combinatorial optimization. Learned candidate rules can accelerate combinatorial optimization, while hard constraints and distribution shift remain central deployment concerns [3, 11]. CLIPPER accepts a replaceable source for its candidate pool. Throughout the experiments, the deterministic score is the demand that each candidate would cover on its own.

3 Problem formulation

This section defines one recorded policy state, its feasible plans, and the coverage objective; Section 4 then describes how CLIPPER computes a plan.

Candidates and demand. Let V be the set of unique candidate facility locations (parking points or areas). Demand is represented by weighted points $d \in \mathcal{D}$ with weights $w_d \geq 0$. Each candidate $e \in V$ covers a subset $C_e \subseteq \mathcal{D}$, such as demand within a walk-access radius. For a selected plan $S \subseteq V$, the weighted union coverage is

$$f(S) = \sum_{d \in \mathcal{D}} w_d \cdot \mathbf{1} \left[d \in \bigcup_{e \in S} C_e \right], \quad (1)$$

where $\mathbf{1}[\cdot]$ is the indicator function. The selector, the audit, and the reported coverage values all derive from f . Monotonicity and submodularity follow directly from the coverage form: the marginal contribution of a candidate is the nonnegative weight of still-uncovered demand in C_e , which can only shrink as the incumbent set grows.

Proposal groups and accounting groups. Pool construction and cap accounting serve different roles and may use different groupings. Each candidate belongs to exactly one *accounting group*, whose cap counts that candidate. The index set $\mathcal{G}_{\text{acc}}$ and assignment $a : V \rightarrow \mathcal{G}_{\text{acc}}$ define the disjoint accounting sets $V_h^{\text{acc}} := \{e \in V : a(e) = h\}$. The *proposal groups* used to form

the candidate pool have index set $\mathcal{G}_{\text{prop}}$ and candidate sets $V_g \subseteq V$, $g \in \mathcal{G}_{\text{prop}}$. Proposal groups may overlap, so the same physical location may enter the pool through more than one group; its accounting assignment remains unique.

Scenario configuration: spatial and policy constraints. The planning scenario Ω records exclusions, locks, spacing, budgets, and accounting-group caps. It specifies the active proposal groups $A(\Omega) \subseteq \mathcal{G}_{\text{prop}}$, forbidden candidates $Z(\Omega) \subseteq V$, and locked facilities $L(\Omega) \subseteq V$. The global facility cap is $B(\Omega)$, and $U_h(\Omega)$ is the cap for accounting group $h \in \mathcal{G}_{\text{acc}}$. Duplicate/conflict classes $\mathcal{H}(\Omega) \subseteq 2^V$ group candidates that describe the same physical curb space; at most one candidate in each class may be chosen. Let $\text{dist}(e_i, e_j)$ denote the configured network distance between two candidates. The minimum permitted distance $d_{\min}(\Omega) \geq 0$ defines the spacing rule. An edit changes one or more of these spatial and policy components, and the resulting scenario Ω is solved and audited as a new state. In the benchmark, the conflict classes are disjoint; the distance rule handles arbitrary pairwise spacing conflicts separately. The candidates available for *new* placements are

$$V^{\text{new}}(\Omega) := \left(\bigcup_{g \in A(\Omega)} V_g \right) \setminus Z(\Omega).$$

Given Ω , let $\mathcal{I}(\Omega)$ denote the feasible family

$$\mathcal{I}(\Omega) := \left\{ S \subseteq V : \begin{array}{l} L(\Omega) \subseteq S, \\ S \setminus L(\Omega) \subseteq V^{\text{new}}(\Omega), \\ |S| \leq B(\Omega), \\ |S \cap V_h^{\text{acc}}| \leq U_h(\Omega) \quad \forall h \in \mathcal{G}_{\text{acc}}, \\ |S \cap H| \leq 1 \quad \forall H \in \mathcal{H}(\Omega), \\ \text{dist}(e_i, e_j) \geq d_{\min}(\Omega) \quad \forall e_i \neq e_j \in S. \end{array} \right\}, \quad (2)$$

and seek $\max_{S \in \mathcal{I}(\Omega)} f(S)$. Under our scenario semantics, *locks override exclusions*. Locked facilities $L(\Omega)$ must appear in every feasible solution even if they fall inside exclusion zones, since they represent existing infrastructure commitments. Exclusions apply only to *new* placements ($S \setminus L(\Omega)$). We assume that the locks themselves form a feasible plan, $L(\Omega) \in \mathcal{I}(\Omega)$. We fix these mandatory locks before analyzing the remaining choices. The residual global budget is $B^L(\Omega) := B(\Omega) - |L(\Omega)|$, and the residual candidate set and feasible family are

$$V^L(\Omega) := V^{\text{new}}(\Omega) \setminus L(\Omega),$$

$$\mathcal{I}^L(\Omega) := \{X \subseteq V^L(\Omega) : L(\Omega) \cup X \in \mathcal{I}(\Omega)\}.$$

The selector operates on $\mathcal{I}^L(\Omega)$ and returns $L(\Omega) \cup X$ for the original problem. In the evaluation, every recorded edit produces a new instance that is solved from the beginning.

Temporal aggregation. Trip records are aggregated into weighted demand points, and the optimizer solves the resulting spatial objective in (1). Temporal utilization, rebalancing, and peak-period service can be analyzed after spatial selection [20, 22, 25]; Section 5 defines the benchmark construction scope used for the tables in this paper.

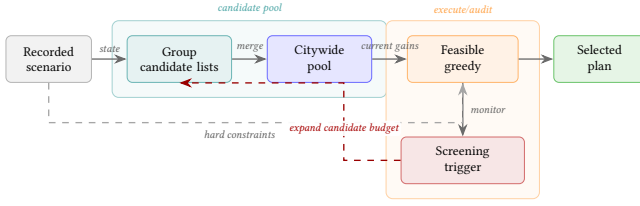


Figure 1: CLIPPER pipeline. Scenario constraints govern both the candidate pool and greedy selection. The screening rule (defined later in this section) can request a larger pool or an offline audit. That audit scans all remaining candidates and calculates the coverage gain omitted from the restricted pool.

4 CLIPPER: optimization over a restricted candidate pool

A planning session poses (2) again after every edit, so the cost that matters is the time to solve one recorded state. CLIPPER can use different rules to decide which candidates the greedy selector evaluates. Its core loop separates three tasks (Fig. 1 and Algorithm 1). First, the candidate rule forms a pool from the active proposal groups. Second, the selector computes each pooled candidate’s exact *current marginal gain* $\Delta(e | S) := f(S \cup \{e\}) - f(S)$: the weight of still-uncovered demand that candidate e would add to the plan S built so far. It adds the candidate with the largest gain among those that satisfy every active constraint. Third, an optional offline audit compares this restricted pool with all remaining feasible candidates. The experiments use deterministic pools ordered by singleton coverage, which is the demand a candidate would cover if selected on its own.

Two operating modes share the same selector. The fixed-width mode CLIPPER-F(K) offers at most K candidates from each active proposal group and uses balanced accounting caps. The adaptive-budget mode CLIPPER-A(B_{prop}) splits a total pool budget of B_{prop} candidates among the proposal groups and uses the coverage-prioritized accounting caps defined below. This pool budget is separate from the facility budget $B(\Omega)$ that bounds the final plan (Eq. (2)). Algorithm 1 covers both modes through a configurable rule for allocating candidate slots; CLIPPER-F sets $K_{g,t} = K$ for every active group.

For each proposal group g , let π_g be its stable candidate order. In round t , the pool rule assigns $K_{g,t}$ slots to group g , which offers a list $P_{g,t}$; the selector evaluates the deduplicated citywide pool $P_t := \bigcup_{g \in A(\Omega)} P_{g,t}$. We write $\mathcal{I}$ for $\mathcal{I}(\Omega)$ where unambiguous. The set S_t contains the residual unlocked selections after t rounds, and $S_t^+ := L(\Omega) \cup S_t$ is the corresponding full solution including locks. Let T be the number of completed selection rounds.

A stop is *truncated* if the pool has no positive feasible action but a full residual scan finds one outside it; otherwise the scan confirms termination. Without that scan, the stop is *unverified*. The experiments report audit-confirmed truncations separately from coverage.

For deterministic replay (re-running a recorded state to reproduce the same plan), the recorded context includes the scenario, demand and candidate data, proposal-group order, candidate ranks,

Algorithm 1 CLIPPER with a configurable rule for allocating candidate slots.

Require: Encoded scenario Ω ; active proposal groups $A(\Omega)$; candidates $\{V_g\}$; stable group orders $\{\pi_g\}$; rule Budget for allocating candidate slots; optional screening rule Trigger

Ensure: Feasible selected plan including mandatory locks

```

1:  $S \leftarrow \emptyset$  ▷ residual selections; locks fixed separately
2:  $B^L(\Omega) \leftarrow B(\Omega) - |L(\Omega)|$  ▷ residual global budget
3: for  $t = 1, 2, \dots, B^L(\Omega)$  do
4:    $\{K_{g,t}\}_{g \in A(\Omega)} \leftarrow \text{Budget}(\Omega, S, t)$ 
5:   for all  $g \in A(\Omega)$  do
6:      $P_{g,t} \leftarrow$  first  $K_{g,t}$  currently valid, unselected candidates in  $\pi_g$ , continuing past skipped candidates
7:   end for
8:    $P_t \leftarrow \bigcup_{g \in A(\Omega)} P_{g,t}$ 
9:    $S^+ \leftarrow L(\Omega) \cup S$ 
10:   $F_t \leftarrow \{e \in P_t : S^+ \cup \{e\} \in \mathcal{I}(\Omega)\}$ 
11:  if  $F_t = \emptyset$  then
12:    Enlarge the pool and restart round  $t$ , or break, marking the stop unverified unless a full residual scan classifies it
13:  end if
14:   $e_t \leftarrow \arg \max_{e \in F_t} [f(S^+ \cup \{e\}) - f(S^+)]$  ▷ first in pool order breaks exact ties
15:  Evaluate Trigger( $\Omega, S^+, P_t$ ) ▷ optional screening rule
16:  if Trigger requests a larger pool then
17:    Enlarge the pool and restart round  $t$ 
18:  end if
19:  if  $f(S^+ \cup \{e_t\}) - f(S^+) \leq 0$  then
20:    Enlarge the pool and restart round  $t$ , or break, marking the stop unverified unless a full residual scan classifies it
21:  end if
22:   $S \leftarrow S \cup \{e_t\}$ 
23: end for
24: return  $L(\Omega) \cup S$ 

```

allocation and cap policies, executor settings, and software version. When gains are exactly equal, the selector takes the first candidate in the deduplicated pool’s deterministic order. This order follows the recorded proposal-group order and the stable rank within each group.

Restricted candidate pools. At each step t , each active proposal group $g \in A(\Omega)$ produces a candidate list $P_{g,t} \subseteq V_g \cap V^L(\Omega)$ with $|P_{g,t}| \leq K_{g,t}$ using a ranking source under a common interface. Here $K_{g,t}$ limits one group’s list; it is not the size of the citywide pool after the lists are combined. For the weighted coverage benchmark, CLIPPER-F and CLIPPER-A rank candidates by their deterministic singleton coverage $\text{pot}_\Omega(e) := f(\{e\})$, restricted to candidates currently feasible under the scenario filters. Because

$$\Delta(e | S) = \sum_{d \in C_e \setminus \bigcup_{j \in S} C_j} w_d \leq \sum_{d \in C_e} w_d = f(\{e\}) = \text{pot}_\Omega(e),$$

the same score gives a deterministic order and is an upper bound on the candidate’s current gain.

To build a group list, the implementation scans each proposal group’s stable rank order. It skips already selected candidates S_{t-1} and candidates that fail the current cap, conflict, or spacing checks against S_{t-1}^+ . It continues through the same order until reaching the requested width or exhausting the group. Thus “top- K ” means the first K valid candidates in the deterministic order: when an invalid candidate is skipped, scanning continues so that its place can be filled. Other ranking sources use the same interface; changing the ranking source leaves the selector, feasibility, and audit definitions unchanged.

Greedy selection over the restricted pool. With residual selections S_{t-1} and the full plan including locks $S_{t-1}^+ := L(\Omega) \cup S_{t-1}$, the algorithm forms the pool $P_t = \bigcup_{g \in A(\Omega)} P_{g,t}$ and selects the best feasible action by evaluating the exact marginal gain in the citywide coverage objective:

$$e_t \in \arg \max_{\substack{e \in P_t \setminus S_{t-1} \\ S_{t-1}^+ \cup \{e\} \in \mathcal{I}}} \Delta(e \mid S_{t-1}^+). \quad (3)$$

Feasibility is enforced by checks derived from Ω (budgets, exclusions, spacing, duplicate/conflict classes). Locked facilities remain included regardless of exclusion zones. If a candidate rule suggests an invalid candidate, the selector rejects it.

Residual view with mandatory locks. Because locks are compulsory elements, the formal analysis is carried out on the residual instance obtained after fixing them. It uses the residual family $\mathcal{I}^L(\Omega)$ from Section 3 and the residual objective $f_\Omega^L(X) := f(L(\Omega) \cup X) - f(L(\Omega))$ on $V^L(\Omega)$. Fixing mandatory elements preserves monotone submodularity. The selector is therefore greedy on the residual instance, initialized at the empty set and adding only unlocked facilities; its reported solution in the original problem is $S_T^+ := L(\Omega) \cup S_T$.

Offline scan of all remaining candidates. Let $S_{t-1}^+ := L(\Omega) \cup S_{t-1}$ denote the full plan after $t-1$ residual selections and let $R_{t-1}(\Omega) := V^L(\Omega) \setminus S_{t-1}$ be the unselected residual candidates. At each round, the audit compares the largest feasible gain among all remaining candidates with the largest gain visible in the restricted pool. Define

$$F_t^{\text{all}} := \{e \in R_{t-1}(\Omega) : S_{t-1}^+ \cup \{e\} \in \mathcal{I}(\Omega)\}, \quad F_t^{\text{pool}} := F_t^{\text{all}} \cap P_t.$$

The two gains are

$$\begin{aligned} m_t^* &:= \max(\{\Delta(e \mid S_{t-1}^+) : e \in F_t^{\text{all}}\} \cup \{0\}), \\ m_t &:= \max(\{\Delta(e \mid S_{t-1}^+) : e \in F_t^{\text{pool}}\} \cup \{0\}). \end{aligned} \quad (4)$$

Including zero defines both quantities even when one of the sets is empty. The difference $\delta_t := m_t^* - m_t$ is the coverage gain omitted from the pool in round t . If $\delta_t = 0$, the pool omits no larger feasible gain. If also $m_t^* > 0$, it contains a candidate attaining that gain; at zero gain, it may be empty. At a terminal check, $m_t = 0 < m_t^*$ means that the pool would stop even though a candidate outside it could still improve coverage. Computing m_t^* requires exact gains for the full remaining set, so this audit is used after a run or at a selected decision checkpoint.

Singleton-coverage screening score. During planning, the screening rule (Trigger in Algorithm 1) can use the less expensive singleton upper bound

$$u_t := \max(\{\text{pot}_\Omega(e) : e \in F_t^{\text{all}}\} \cup \{0\}),$$

which satisfies $u_t \geq m_t^*$. Therefore $\delta_t \leq \hat{\delta}_t^{\text{ub}} := u_t - m_t$. This screening score is a conservative upper bound on the gain omitted in the current round and can trigger a larger pool or an offline audit. It does not bound the coverage difference between the final plans. To certify a stop before exhausting the residual budget, verify $u_{T+1} \leq 0$ or scan all remaining candidates. For offline reporting, the audit can sum δ_t over all audited rounds.

Operating policies: balanced caps and coverage-prioritized caps. The *balanced-cap policy* gives every accounting group the benchmark's equal cap and is used by CLIPPER-F and its corresponding full-set greedy run. The planner-specified *coverage-prioritized policy* keeps the global budget unchanged but assigns more of it to groups with more active candidates. After subtracting locks, largest-remainder rounding divides the remaining budget in proportion to active candidate counts. With multiplier $\lambda = 1.5$, CLIPPER-A records the cap $U_h^A = \ell_h + \lceil \lambda b_h \rceil$, where ℓ_h is the number of locked facilities and b_h is that group's rounded share. These caps are stored in the scenario before selection and enforced as hard constraints. CLIPPER-A and its corresponding full-set greedy run both use them.

Budgeted adaptive candidate policy. CLIPPER-A keeps the deterministic singleton-coverage ranking rule and exact greedy selection, and replaces the fixed number per group with B_{prop} total slots per round. Every selected candidate e is charged to its accounting assignment $a(e)$. For allocating slots in the evaluated benchmark, each proposal group g is aligned with one accounting group $h(g)$, with $V_g \subseteq V_{h(g)}^{\text{acc}}$. A group receives no more slots once that cap is full. Among groups whose caps are not full, the allocation depends on the number of feasible candidates and the highest singleton score. It first reserves the configured minimum number of slots for each such group, then uses largest-remainder rounding and resolves ties by recorded group order. We evaluate $B_{\text{prop}} \in \{4096, 8192\}$; the main CLIPPER-F comparison uses $K = 1024$ (Table 3). Both modes share the objective, data, selector, and constraint checks but use the policies and slot allocations stated above.

Complexity and cost comparison. Let $c := \max_{e \in V^L(\Omega)} |C_e|$ be the largest demand support of one candidate. The number of completed rounds satisfies $T \leq B^L(\Omega)$. If N_t candidates reach the selector in round t , CLIPPER performs $O(c \sum_{t=1}^T N_t)$ work for gain calculations. For CLIPPER-F this is $O(TG_{\text{eff}}Kc)$, where G_{eff} is the number of active groups; for CLIPPER-A it is $O(TB_{\text{prop}}c)$. Full-set greedy uses $O(T|V^L(\Omega)|c)$. If the pool contains every remaining feasible candidate in every round, $\delta_t = 0$ and CLIPPER returns the same greedy sequence under the stated tie rule.

5 Empirical evaluation

CLIPPER solves each recorded policy state from the beginning; it carries neither selected sites nor candidate scores over from the previous edit. Each primary comparison uses full-set greedy with the same cap policy as the corresponding CLIPPER mode; we call this run the *control*. Specifically, CLIPPER-F and its control use balanced caps, while CLIPPER-A and its control use coverage-prioritized caps. A coverage difference within either pair can therefore be attributed to the restricted candidate pool rather than to different constraints. Every method is rerun on every state in the complete $E_0, \dots, E_{10}$ chain (Table 3). Figure 2 shows one such edit on a separate illustrative Braunschweig diagnostic: between E_0 and E_5 , exclusions remove candidates, locks persist from the baseline, and the replanned set retains most baseline sites while replacing others.

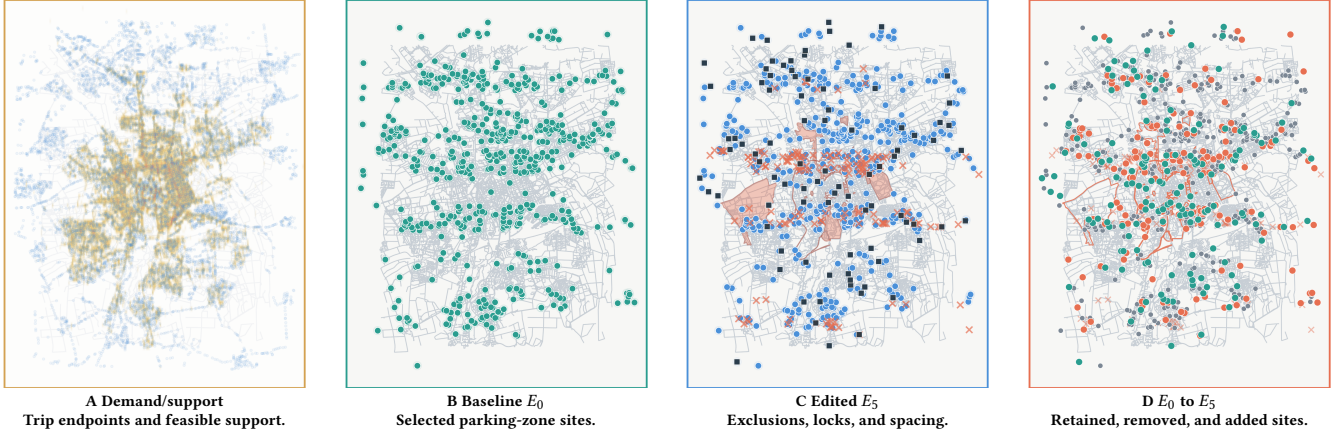


Figure 2: Illustrative Braunschweig diagnostic on a separate 25 m grid; quantitative comparisons use the 10 m support in Section 5.1. Panel A shows demand and candidates; B marks baseline sites in teal; C shows selected sites (blue), locks (dark squares), and exclusions (coral); D marks retained, removed, and added sites in gray, coral, and teal. At E_5 , 162 candidates are forbidden, 90 sites are locked, and 25 m spacing is active; the plan retains 443 baseline sites and adds 157 replacements.

5.1 Setup

Data construction. The benchmark retains every valid record in three non-public operator trip feeds available to the project for internal shared-mobility research. For the paper-facing objective, origins and destinations are snapped to spatial support and aggregated as endpoint counts; individual trip records, coordinates, timestamps, and vehicle or provider identifiers remain non-public and are not released. After validation, the feeds contain 243 649 Braunschweig trips (2024-01-01–2024-04-30), 4 958 963 Munich trips (2023-12-18–2024-08-22), and 12 306 947 Berlin trips over the latter interval. We snap each valid origin and destination to its nearest feasible support point and assign one unit of demand to each snapped endpoint; temporal bins are summed for the paper’s spatial objective.

Candidate grids use OpenStreetMap (OSM)-derived sidewalk, footway, and public-parking support [8] after restricted-area filtering, at 10 m resolution in Braunschweig and 25 m in Munich and Berlin. We first fit k -means with 16 clusters (seed 42), then split or merge clusters until each proposal group contains 1,750–2,750 candidate points. This yields 29 proposal groups over 60 495 unique candidates in Braunschweig, 16 groups over 32 907 candidates in Munich, and 36 groups over 68 922 candidates in Berlin. Table 1 summarizes the three instances. The same synthetic partitions serve as proposal and accounting groups in the benchmark. Each proposal group is therefore paired one-to-one with the accounting group that contains its candidates, as required by CLIPPER-A’s slot-allocation rule. The groups are synthetic spatial partitions rather than administrative districts. No demand or candidate downsampling is used.

Spatial objective and constraints. All main-text methods use weighted point coverage within a 200 m OSM walk-network radius. For a given state, they share the complete demand and candidate data, encoded locks, exclusions, conflict classes, spacing rules, and global facility budget. Coverage incidence and facility spacing use shortest paths on per-city OSM walk graphs. In Table 3, rows

City	Trips	Cand.	Budget	Groups	F slots
Braunschweig	243 649	60 495	600	29	29 696
Munich	4 958 963	32 907	1200	16	16 384
Berlin	12 306 947	68 922	1600	36	36 864

Table 1: Full-data instances. F slots are the maximum CLIPPER-F($K = 1024$) entries per round before filtering and deduplication; CLIPPER-A uses 8192 total slots.

in the balanced-cap block use equal caps for all accounting groups; rows in the coverage-prioritized block use $\lambda = 1.5$ and leave the global budget unchanged. Each CLIPPER mode and its control use the same caps.

Constructed edit chain. The $E_0, \dots, E_{10}$ chain is a controlled stress test whose complete configuration appears in Table 2. Starting from the unrestricted E_0 state, later states place increasing shares of the most consequential baseline sites inside 75 m exclusion zones and retain increasing shares as mandatory locks. The locks form a growing prefix of baseline sites ordered by their coverage contribution and spatial dispersion. From E_5 , the states add a minimum network distance between selected sites. From E_8 , they also add 150 m exclusion zones centered on dense endpoint cells; the scenario records these halos as additional exclusions.

Compared methods. The primary suite compares CLIPPER-F(1024) and CLIPPER-A($B_{\text{prop}} \in \{4096, 8192\}$) with full-set greedy under the same policy. The auxiliary controls are competitive local search [13], stochastic greedy [15], and thresholded greedy [2].

Audit of the fixed candidate order. For CLIPPER-F, a separate exact audit scans every remaining feasible candidate after each selection for $K \in \{256, 512, 1024\}$, in all 11 states and without downsampling. It also performs one terminal check when the pool has no positive feasible action. Let $\mathcal{T}_{\text{audit}}$ contain these audited rounds. In round t , m_t^* is the largest feasible gain found by the full scan, while m_t is the largest gain among candidates in the pool (Section 4).

State	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}
Baseline sites inside 75 m exclusions (%)	0.0	2.5	5.0	5.0	7.5	7.5	10.0	12.5	12.5	15.0	17.5
Baseline sites kept as locks (%)	0.0	0.0	0.0	10.0	10.0	15.0	20.0	20.0	25.0	30.0	35.0
150 m exclusion halos	0	0	0	0	0	0	0	0	10	15	20
Minimum network spacing (m)	0	0	0	0	0	25	35	45	50	55	60

Table 2: Complete configuration of the controlled edit chain. Exclusions target a growing prefix of baseline sites ordered by individual coverage contribution; locks use contribution and spatial dispersion. Each halo is an exclusion zone centered on a dense endpoint cell.

We record whether their difference is at most $\max(10^{-9}, 10^{-6}|m_t^*|)$. The reported percentage is therefore the share of audited rounds in which the pool omits no larger feasible gain within this tolerance, including terminal checks with both gains zero. This compares gains, not candidate identities. We also sum $\delta_t = m_t^* - m_t$ over $\mathcal{T}_{\text{audit}}$ and divide by the coverage added after mandatory locks. This second quantity reports the size of the gains omitted in individual rounds relative to the coverage added by the run.

Comparison with scores updated after each selection. The fixed rule computes singleton coverage once at the start of a state. The comparison rule starts from the same scores, builds an index from each demand point to the candidates that cover it, and updates only candidates affected by demand that has just become covered. The heap for each group then exposes its current top $K = 1024$ candidates. Both rules use the same Python executor, which selects one candidate per step, as well as the same data, policy checks, and stopping rule. Their timers include score initialization, updates, heap maintenance, pool construction, and selection; audits are disabled. These paired runs use the same host as the primary comparison. Panel (b) of Table 4 compares the two rules within this executor, whereas panel (a) reports the separate compiled exact-audit execution. Their fixed-order coverage values differ slightly; the paired comparison uses only panel (b)’s fixed-order reference.

Documented municipal policy inputs. We additionally convert two official Braunschweig open-data resource distributions of e-scooter no-parking zones, dated 2025-01-28 and 2026-02-17, into exclusion-only states [24]. The CC BY 4.0 archives are pinned by resource ID and SHA-256 digest. We repair and union the polygons in EPSG:25832, reproject the complete 60 495-candidate support, and mark every intersecting candidate as forbidden. These standalone snapshots use the same demand and budget with no locks or spacing, isolating how the documented policy geometries change feasibility and the returned plan.

Metrics and timing boundary. Let $W = \sum_{d \in \mathcal{D}} w_d$ be total demand weight. Coverage percentage is $100f(S)/W$: the share of trip endpoints that lie within the walk radius of some selected site. Coverage gaps subtract method coverage from the corresponding control. A gap of 0.245 percentage points means that the plan covers 0.245 percentage points less of total demand weight than the control. Thus the gap measures agreement with an empirical greedy reference rather than global optimality. All primary and paired score-update timings use the same Intel Core Ultra 9 285K host. In Table 3, CLIPPER and full-set greedy share the rollout boundary: the timer starts after scenario initialization and includes per-round pool construction, exact marginal evaluation, feasibility

checks, and selection. It excludes scenario construction, preliminary validation of candidate-pool dimensions, raw preprocessing, optional screening and exact auditing, and output serialization. Optimization and mixed-integer linear programming (MILP) diagnostics report their own broader boundaries separately.

5.2 Results

5.2.1 Full-city comparison under the same policy (Q1). Table 3 reports means across all 11 states. The mean coverage gap for CLIPPER-F(1024) is 0.245 percentage points in Braunschweig, 0.003 in Munich, and 0.001 in Berlin. Mean rollout time falls from 24.029 to 1.762 seconds in Braunschweig, 22.943 to 1.493 in Munich, and 52.685 to 1.825 in Berlin. The corresponding speedups are $13.63\times$ in Braunschweig, $15.37\times$ in Munich, and $28.87\times$ in Berlin.

Summed over the 11 states of one city, the tabulated means correspond to 16.4–20.1 seconds for CLIPPER-F and 4.2–9.7 minutes for its control (Table 3). Per-state results show that these speedups persist across the chain. The coverage gaps range from -0.038 to 0.377 percentage points in Braunschweig, from -0.010 to 0.016 in Munich, and from -0.001 to 0.002 in Berlin. The corresponding speedups range from 7.4 to $40.7\times$, from 8.4 to $49.1\times$, and from 15.7 to $84.3\times$, respectively. Speedups are largest before spacing activates at E_5 , but remain at least $7.4\times$ once spacing and the exclusion halos are combined. The small negative gaps occur because the two greedy runs can follow different selection sequences and end in different feasible plans.

CLIPPER-A. Under the coverage-prioritized cap policy, CLIPPER-A(8192) has gaps to its control of 1.82 percentage points in Braunschweig, 0.12 in Munich, and 0.27 in Berlin. Its rollout time is 0.11, 0.15, and 0.09 times the corresponding control, respectively. CLIPPER-A and CLIPPER-F differ in both cap policy and pool allocation, so differences between the modes reflect both choices.

Controls and optimization checks. Two additional cap-policy ablations support this reading: adaptive pool budgets under balanced caps remain near the control, and under coverage-prioritized caps fixed per-group pools underperform CLIPPER-A(8192) by roughly 14.1 percentage points in Braunschweig, 2.8 in Munich, and 4.9 in Berlin. Competitive local search reaches similar coverage but requires 22.4–72.3 seconds under its reported boundary; stochastic and thresholded variants are slower, lower-quality, or both. Further supporting diagnostics are exact E_0 solves with small facility budgets and mixed-integer solves over the restricted pool with the full facility budget. They distinguish closeness to full-set greedy from closeness to an optimum.

Method / control	Mean coverage (%)			Mean gap to control (pp)			Mean rollout (s)			Rollout/control		
	BS	MUC	BER	BS	MUC	BER	BS	MUC	BER	BS	MUC	BER
<i>Balanced caps: full-set greedy and CLIPPER-F</i>												
Full-set greedy	90.4	69.7	59.5	0.000	0.000	0.000	24.029	22.943	52.685	1.000	1.000	1.000
CLIPPER-F(1024)	90.1	69.7	59.5	0.245	0.003	0.001	1.762	1.493	1.825	0.073	0.065	0.035
<i>Coverage-prioritized caps: full-set greedy and CLIPPER-A</i>												
Full-set greedy	96.8	81.2	69.5	0.000	0.000	0.000	28.343	22.931	48.826	1.000	1.000	1.000
CLIPPER-A(4096)	89.8	78.2	66.1	6.920	3.010	3.410	3.222	3.383	4.393	0.114	0.148	0.090
CLIPPER-A(8192)	95.0	81.1	69.3	1.820	0.120	0.270	3.219	3.360	4.415	0.114	0.147	0.090

Table 3: Main comparison on complete data, averaged over $E_0, \dots, E_{10}$. BS, MUC, and BER denote the three cities; pp denotes percentage points. Each block pairs CLIPPER with full-set greedy under the same caps, on the same host and with the same rollout timing boundary (Section 5.1). Gap is control minus method coverage; rollout/control is their time ratio.

(a) Audit of a candidate order computed once per state							
City	K	Mean coverage (%)	Audited rounds: no larger feasible gain omitted (%)	Omitted gains / added coverage	Runs stopped by pool limit (of 11)		
Braunschweig	256	77.248	4.4	3.295	11/11		
	512	85.677	13.3	1.071	11/11		
	1024	90.132	51.0	0.140	0/11		
Munich	256	64.623	10.5	0.748	6/11		
	512	69.395	66.4	0.046	0/11		
	1024	69.685	100.0	0.000	0/11		
Berlin	256	58.638	24.4	0.214	0/11		
	512	59.542	95.6	0.002	0/11		
	1024	59.546	100.0	0.000	0/11		
(b) Scores fixed at state start versus scores updated after each selection, K = 1024							
City	Fixed scores: mean cov. (%)	Updated scores: mean cov. (%)	Coverage change (pp)	Fixed scores: median wall (s)	Updated scores: median wall (s)	Updated / fixed wall time	Score rows updated (%)
Braunschweig	90.172	90.375	+0.2037	24.04	61.08	2.75	0.128
Munich	69.678	69.683	+0.0049	39.09	74.73	1.76	0.050
Berlin	59.547	59.546	−0.0006	68.75	152.85	2.32	0.026

Table 4: Results on $E_0, \dots, E_{10}$. Panel (a) scans the full remaining set after each selection and at termination; it reports rounds with no larger feasible gain omitted, normalized omitted gains, and stops caused by the pool limit. Panel (b) uses the stepwise executor from Section 5.1; its timer includes initialization, updates, heap maintenance, pool construction, and selection. The host is shared with Table 3, but the execution path and timer differ; panel (b)’s times are compared only within that panel. Coverage values are means; wall times and paired per-state time ratios are medians. The final column normalizes post-initialization score updates by the product of selection calls and stored candidate rows.

5.2.2 A fixed candidate order and scores updated during selection (Q2). CLIPPER-F ranks each group’s candidates once per state by singleton coverage and then reuses that order. As sites are added, candidates cover overlapping demand and their current gains change. The audit in Table 4 asks whether the restricted pool omits any larger feasible gain found by scanning all remaining candidates. With $K = 256$, no such gain is omitted in 4.4% of audited rounds in Braunschweig, 10.5% in Munich, and 24.4% in Berlin. The percentages rise to 13.3%, 66.4%, and 95.6% at $K = 512$, and to 51.0%, 100.0%, and 100.0% at $K = 1024$, respectively. At $K = 1024$, no audited run terminates while a candidate outside the pool could still increase coverage. The sum of omitted gains divided by coverage added after locks is 0.140 in Braunschweig and 0 in Munich and Berlin.

The round-by-round percentages and final coverage answer different questions. In Braunschweig, the $K = 1024$ pool omits no larger feasible gain in 51% of audited rounds, yet its mean final coverage over the eleven states is only 0.245 percentage points below the full-set control. A later selection can cover demand omitted in an earlier round. The result shows that an order computed once

can cease to identify the largest current gains without causing an early stop or a similarly large change in final coverage.

Within the stepwise executor used for the paired comparison, updating affected scores changes mean coverage by +0.2037 percentage points in Braunschweig, +0.0049 in Munich, and -0.0006 in Berlin. Braunschweig is the only city that changes by more than 0.01 percentage points and the only one where the exact audit finds omitted gains at $K = 1024$. After initialization, the update rule touches only 0.128% of stored candidate rows in Braunschweig, 0.050% in Munich, and 0.026% in Berlin. Maintaining and reading the heaps raises the median paired wall-time ratio to 2.75 $\times$, 1.76 $\times$, and 2.32 $\times$, respectively. Updating scores thus provides a measurable coverage benefit in Braunschweig and little or none in the other two cities, while requiring more time in all three (Table 4).

5.2.3 Documented policy-input case (Q3). Table 5 uses two official resource versions instead of constructed exclusions. The polygon union grows by 3.154 km²; the number of candidates inside an exclusion grows from 1435 (2.37%) to 4653 (7.69%). This yields 3218 newly forbidden candidates and no candidate that becomes feasible again.

Both rules stop below the 600-site budget because the terminal full-set check finds no positive feasible gain: after 540 selections with fixed scores and 541 with updated scores. With fixed scores, CLIPPER-F retains 510 of 540 selected sites; with updated scores, it retains 511 of 541. Both rules replace 30 sites: 22 were directly made infeasible by the newer resource geometry, while eight remained feasible and changed later in the selection sequence. Coverage changes by -0.094 and -0.106 percentage points, respectively. Similar coverage therefore does not mean a similar plan: judged by the objective alone the two official policy versions look almost interchangeable, yet their plans differ in 30 of about 540 sites that still require municipal assessment, implementation, and communication.

5.2.4 Recorded audit and repeatability checks. For each audited round, the implementation computes the largest feasible gain in the full remaining set, the largest gain in the restricted pool, their difference, and the terminal check. Panel (b) disables these scans so that it measures only the two candidate-ordering rules in the stepwise executor.

In a separate full-data diagnostic of CLIPPER-F(1024) using the non-public inputs, three reruns of every state and city (99 runs) produced identical coverage, step counts, numbers of gain calculations, termination reasons, and terminal flags within each state. Every run ended with no feasible candidate that could add coverage, and none stopped because its candidate pool was too narrow. These checks and the municipal source manifest are retained with the project records.

6 Municipal implementation pathway

We propose a municipal process for comparing policy alternatives using CLIPPER’s optimization and audit functions (Table 6). In the Braunschweig case, the plans differ in 30 sites for only about 0.1 percentage points of coverage, so the interface must show site changes beside coverage. At $K = 1024$, the pool omits no larger feasible gain in every audited Munich and Berlin round but in 51% of Braunschweig rounds; planners therefore need to calibrate score updates locally (Tables 4 and 5). To support this calibration and the final decision, we propose scanning every remaining candidate and recording what the pool omitted.

(a) Policy resources and resulting coverage					
Resource date	Zones	Area (km ²)	Forbidden cand.	Fixed scores: cov. (%)	Updated scores: cov. (%)
2025-01-28	51	6.275	1435	91.609	91.807
2026-02-17	55	9.429	4653	91.515	91.701

(b) Change in selected sites from the first resource to the second					
Candidate scores	Selected old/new	Retained	Removed	Removed after becoming newly forbidden	Added
Fixed at state start	540/540	510	30		22
Updated after selection	541/541	511	30		22

Table 5: Braunschweig case with two official no-parking-zone resources and all 60 495 candidates. Both runs use $K = 1024$, identical demand, a 600-site budget, and no locks or spacing; of 30 removed sites, 22 became forbidden and eight remained feasible.

The collaboration established the workflow requirements. In the Braunschweig case, we registered two official policy versions and compared the resulting plans (Table 5). In the proposed pilot, a municipal policy owner defines constraints and acceptance checks, and a data steward validates demand, candidates, and the network. Each edit produces a fresh scenario Ω ; the interface shows coverage and the sites retained, removed, or added. The chosen state then receives the final scan and archive package in Table 6. Following planning-support practice [17, 19], evaluation should compare full-set and CLIPPER-assisted sessions and measure time to an accepted scenario, alternatives inspected, constraint corrections, final coverage, and the usefulness of the change and audit records.

7 Limitations

Evidence about planning use. The Braunschweig collaboration informed requirements; the experiments evaluate optimizer behavior rather than user interaction or deployment outcomes. Without recorded planning sessions or edit logs, the study does not estimate edit frequency, acceptable response times, alternatives considered, or planner utility. Reported speedups cover optimization rather than end-to-end deliberation.

Model, data, and transfer. The experiments use the same partitions for proposal groups and accounting caps; overlapping proposal groups were not evaluated. The weighted point-coverage model does not represent curb-segment capacity, facility congestion, user compliance, equity outcomes, temporal rebalancing, operator behavior, or institutional approval. Addressing these dimensions requires assignment, occupancy, or additional utility and feasibility inputs. The three trip feeds are non-public, incomplete market samples, and a held-out city-provider pair is needed to assess transfer beyond them. Independent reconstruction additionally requires the processed candidates, recorded scenarios, and fixed OSM snapshots; aggregate paper tables alone are insufficient. In this paper, replay means deterministic execution of a versioned scenario bundle within its recorded software environment. It is distinct from public reproduction because the operator trip feeds cannot be released.

Runtime comparisons. Speedups compare implementations within the stated timing boundary, without separating pool restriction from execution technique. A comparison with classical lazy greedy under matched execution and timing conditions remains open.

Stage	Inputs, outputs, and acceptance check
Register	<i>Input:</i> policy geometry, candidates, demand, coordinate system, validity date. <i>Check:</i> source checksum, valid geometry, and candidate intersections.
Calibrate	<i>Input:</i> representative recorded states and local time/quality tolerances. <i>Check:</i> choose K , cap policy, and candidate rule; meet local targets and scan for improving candidates outside the pool when a run stops.
Compare	<i>Input:</i> exclusions, locks, spacing, caps, and facility budget for each alternative. <i>Check:</i> feasible plan, coverage, changed sites, terminal status, and input/output checksums.
Final decision	<i>Input:</i> chosen policy state and audit level. <i>Output and check:</i> scan all remaining candidates, archive inputs and outputs, and obtain policy-owner sign-off.

Table 6: Municipal pilot pathway. The municipality owns policy and acceptance thresholds; CLIPPER supplies plans, comparisons, and audit records.

Candidate scores. Both the fixed and updated candidate orders restart after each edit (Table 4). Learned or capacity-aware candidate rules can use the same selector, but require validation on unseen data and checks for bias [14].

8 Conclusion

Across the three cities, CLIPPER-F’s mean coverage gap over the complete chain is at most 0.245 percentage points while its mean rollout is 13.6–28.9× faster than full-set greedy under the same policy (Table 3). Both CLIPPER modes enforce every encoded hard constraint and replay recorded states deterministically. At $K = 1024$, none of the 11 audited runs per city ends because of the pool limit, while the score-update comparison shows the local trade-off between coverage and computation time (Table 4).

These functions serve distinct planning tasks: the optimizer computes feasible alternatives, the audit measures omitted gains, and comparisons between policy versions identify changed sites. The Braunschweig plans differ in 30 sites despite similar coverage (Table 5), so aggregate coverage alone cannot describe what a municipality would implement. We propose using these functions to compare alternatives, calibrate score updates locally, and scan the full candidate set before a final decision. The archive retains the policy inputs and selected sites.

Acknowledgments

We thank our partners at the City of Braunschweig for contributing operational requirements. Generative AI tools were used only for editing and language polishing. The authors take full responsibility for the scientific content, experiments, analyses, and conclusions.

References

- [1] Arpit Agarwal and Eric Balkanski. 2024. Learning-Augmented Dynamic Submodular Maximization. In *Advances in Neural Information Processing Systems*, Amir Globerson, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrike Paquet, Jakub Tomczak, and Cheng Zhang (Eds.), Vol. 37. Curran Associates, Inc., Vancouver, 14148–14176. doi:10.52202/079017-0453 NeurIPS 2024; earlier preprint arXiv:2311.13006.
- [2] Ashwinkumar Badanidiyuru and Jan Vondrák. 2013. Fast Algorithms for Maximizing Submodular Functions. In *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM, Philadelphia, 1497–1514. doi:10.1137/1.9781611973402.110
- [3] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. 2021. Machine learning for combinatorial optimization: a methodological tour d’horizon. *European Journal of Operational Research* 290, 2 (2021), 405–421. doi:10.1016/j.ejor.2020.07.063
- [4] Yutong Cai, Ghim Ping Ong, and Qiang Meng. 2023. Bicycle sharing station planning: From free-floating to geo-fencing. *Transportation Research Part C: Emerging Technologies* 147 (2023), 103990. doi:10.1016/j.trc.2022.103990
- [5] Richard Church and Charles ReVelle. 1974. The Maximal Covering Location Problem. *Papers of the Regional Science Association* 32, 1 (Dec. 1974), 101–118. doi:10.1007/BF01942293
- [6] Juan Carlos Garcia-Palomares, Javier Gutiérrez, and Marta Latorre. 2012. Optimizing the location of stations in bike-sharing programs: A GIS approach. *Applied Geography* 35, 1-2 (2012), 235–246. doi:10.1016/j.apgeog.2012.07.002
- [7] Stan Geertman, Joseph Jr. Ferreira, Robert Goodspeed, and John Stillwell (Eds.). 2015. *Planning Support Systems and Smart Cities*. Springer, Cham. doi:10.1007/978-3-319-18368-8
- [8] Mordechai Haklay and Patrick Weber. 2008. OpenStreetMap: User-Generated Street Maps. *IEEE Pervasive Computing* 7, 4 (2008), 12–18. doi:10.1109/MPRV.2008.80
- [9] Nicholas Klein, Anne Brown, and Calvin Thigpen. 2023. Clutter and Compliance: Scooter Parking Interventions and Perceptions. *Active Travel Studies* 3, 1 (2023), 1–21. doi:10.16997/ats.1196
- [10] Richard E. Klosterman. 1997. Planning Support Systems: A New Perspective on Computer-Aided Planning. *Journal of Planning Education and Research* 17, 1 (1997), 45–54. doi:10.1177/0739456X9701700105
- [11] James Kotary, Ferdinando Fioretto, Pascal Van Hentenryck, and Bryan Wilder. 2021. End-to-End Constrained Optimization Learning: A Survey. In *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, Zhi-Hua Zhou (Ed.). International Joint Conferences on Artificial Intelligence Organization, Virtual, 4475–4482. doi:10.24963/ijcai.2021/610
- [12] Andreas Krause and Daniel Golovin. 2014. Submodular Function Maximization. In *Tractability: Practical Approaches to Hard Problems*, Lucas Bordeaux, Youssef Hamadi, and Pushmeet Kohli (Eds.). Cambridge University Press, Cambridge, 71–104. doi:10.1017/CBO9781139177801.004
- [13] Jon Lee, Maxim Sviridenko, and Jan Vondrák. 2010. Submodular Maximization over Multiple Matroids via Generalized Exchange Properties. *Mathematics of Operations Research* 35, 4 (2010), 795–806. doi:10.1287/moor.1100.0463
- [14] Ninareh Mehrabi, Fred Morstatter, Nripsuta Saxena, Kristina Lerman, and Aram Galstyan. 2021. A Survey on Bias and Fairness in Machine Learning. *Comput. Surveys* 54, 6 (2021), 1–35. doi:10.1145/3457607
- [15] Baharan Mirzasoleiman, Ashwinkumar Badanidiyuru, Amin Karbasi, Jan Vondrák, and Andreas Krause. 2015. Lazier Than Lazy Greedy. *Proceedings of the AAAI Conference on Artificial Intelligence* 29, 1 (2015), 1812–1818. doi:10.1609/aaai.v29i1.9486
- [16] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. 1978. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming* 14, 1 (Dec. 1978), 265–294. doi:10.1007/BF01588971
- [17] Peter Pelzer, Stan Geertman, Rob van der Heijden, and Étienne Rouwette. 2014. The added value of Planning Support Systems: A practitioner’s perspective. *Computers, Environment and Urban Systems* 48 (2014), 16–27. doi:10.1016/j.compenvurbsys.2014.05.002
- [18] Christopher Pettit, Ashley Bakelmun, Scott N. Lieske, Stephen Glackin, Karlson Hargroves, Giles Thomson, Heather Shearer, Hussein Dia, and Peter Newman. 2018. Planning support systems for smart cities. *City, Culture and Society* 12 (2018), 13–24. doi:10.1016/j.ccs.2017.10.002
- [19] Patrizia Russo, Rosa Lanzilotti, Maria F. Costabile, and Christopher J. Pettit. 2018. Towards satisfying practitioners in using Planning Support Systems. *Computers, Environment and Urban Systems* 67 (2018), 9–20. doi:10.1016/j.compenvurbsys.2017.08.009
- [20] Sören Schleibaum, Julian Teusch, Yannick Oskar Scherr, and Jörg P. Müller. 2025. Spatial-Temporal Patterns of E-Scooter Demand Prediction Across Cities. *Transportation Research Procedia* 86 (2025), 48–55. doi:10.1016/j.trpro.2025.04.007
- [21] Stefan Schwerdfeger, Stefan Bock, Nils Boysen, and Dirk Briskorn. 2025. Manage the Curb: Optimization of Time-Varying Parking Zones in Micromobility Systems. *Transportation Science* 59, 5 (2025), 972–989. doi:10.1287/trsc.2024.0855
- [22] Susan Shaheen, Adam Cohen, and Jacquelyn Broader. 2022. Shared micromobility: Policy, practices, and emerging futures. In *Routledge Companion to Cycling*. Routledge, London, 254–262. doi:10.4324/9781003142041-33
- [23] Stadt Braunschweig. 2021. Qualitätsvereinbarung: Vereinbarung über die Qualität des Angebots von Elektrokleinstfahrzeugen in Braunschweig. Municipal quality agreement. https://www.braunschweig.de/leben/stadtplan_verkehr/sharing_Angbote/E-Scooter_Qualitaetsvereinbarung.pdf Accessed 2026-08-10.
- [24] Stadt Braunschweig. 2026. E-Tretroller | Parkverbotszonen. Municipal open-data dataset. <https://opendata.braunschweig.de/dataset/e-tretroller-parkverbotszonen> CC BY 4.0; resource distributions dated 2025-01-28 and 2026-02-17; accessed 2026-08-24.
- [25] Julian Teusch, Jan Niklas Gremmel, Christian Koetsier, Fatema Tuj Johora, Monika Sester, David M. Woisetschlager, and Jörg P. Müller. 2023. A Systematic Literature Review on Machine Learning in Shared Mobility. *IEEE Open Journal of Intelligent Transportation Systems* 4 (2023), 870–899. doi:10.1109/OJITS.2023.3334393
- [26] Julian Teusch, Jörg Philipp Müller, and Monika Sester. 2026. CLIPPER: Re-playable Shortlisted Optimization for Repeated Spatial Coverage Planning. In *Proceedings of the 34th ACM International Conference on Advances in Geographic Information Systems*. ACM, New York, NY, USA. doi:10.1145/3841645.3843432
- [27] Julian Teusch, Bruno Neumann Saavedra, Yannick Oskar Scherr, and Jörg P. Müller. 2025. Strategic planning of geo-fenced micro-mobility facilities using reinforcement learning. *Transportation Research Part E: Logistics and Transportation Review* 194 (Feb. 2025), 103872. doi:10.1016/j.tre.2024.103872
- [28] Zehui Yin, Greg Rybarczyk, Anran Zheng, Lin Su, Bingrong Sun, and Xiang Yan. 2024. Shared micromobility as a first- and last-mile transit solution? Spatiotemporal insights from a novel dataset. *Journal of Transport Geography* 114 (2024), 103778. doi:10.1016/j.jtrangeo.2023.103778
- [29] Jiawei Zhang, Xiao Pan, Moyin Li, and Philip S. Yu. 2016. Bicycle-sharing systems expansion: station re-deployment through crowd planning. In *Proceedings of the 24th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (SIGSPATIAL ’16)*. ACM, New York, NY, USA, 1–10. doi:10.1145/2996913.2996926
- [30] De Zhao and Ghim Ping Ong. 2021. Geo-fenced parking spaces identification for free-floating bicycle sharing system. *Transportation Research Part A: Policy and Practice* 148 (2021), 49–63. doi:10.1016/j.tra.2021.03.007